

TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN HỌC PHẦN XÂY DỰNG PHẦN MỀM THEO MÔ HÌNH PHÂN LỚP

Giảng viên phụ trách: Từ Lăng Phiêu

Sinh viên thực hiện:

Mao Thị Minh Hằng	3122410102
Nguyễn Thị Thanh Hằng	3122410103
Trần Bảo Hân	3122410105
Nguyễn Thùy Trang	3122410417
Phạm Duy Trục	3122410439

Email liên hệ: phamduytruc04@gmail.com

Mục lục

Chương 1: Giới thiệu	3
I. Tổng quan và mục tiêu dự án	3
II. Phát biểu vấn đề	4
III. Phạm vi dự án	4
Chương 2: Cơ sở lý thuyết	6
I. Công nghệ và Thư viện sử dụng	6
1. Ngôn ngữ lập trình	6
2. Framework và Thư viện Backend	6
3. Frontend	6
4. Cơ sở dữ liệu	6
5. Giao tiếp giữa các service	7
6. Triển khai hệ thống	7
II. Kiến trúc hệ thống	7
III. Thiết kế cơ sở dữ liệu	9
1. Sơ đồ cơ sở dữ liệu (sơ đồ ER)	9
2. Các bảng và mối quan hệ	9
3. Phương pháp mô hình hóa dữ liệu	10
Chương 3: Hiện thực	11
I. Triển khai các tính năng	11
1. Message Service	11
2. Friend Service	11
3. Search History Service	12
4. User Service	12
5. Post Service	13
II. Ảnh chụp màn hình và minh họa	13
III. Cấu trúc mã nguồn	19
1. Tổ chức thư mục	19
1.1 Cấu trúc thư mục <code>frontend</code>	19
1.2 Cấu trúc thư mục <code>friend_service</code>	20
1.3 Cấu trúc thư mục <code>post_service</code>	21
1.4 Cấu trúc thư mục <code>search_service</code>	21
1.5 Cấu trúc thư mục <code>user_service</code>	22
2. Các thành phần chính và tương tác	22
3. Flowchart cho các quy trình phức tạp	23
3.1 Gửi yêu cầu kết bạn (Send Request)	23
3.2 Xác thực token và lấy thông tin người dùng (Get User by Token)	25

3.3	Tạo cuộc hội thoại (Create Conversation)	26
3.4	Gửi tin nhắn (Send Message)	27
3.5	Thêm bạn bè (Add Friend)	28
Chương 4: Cài đặt và triển khai		30
I.	Yêu cầu hệ thống	30
II.	Các bước cài đặt	30
Chương 5: Đóng góp của thành viên		32
I.	Phân công nhiệm vụ	32
II.	Đóng góp chi tiết của từng thành viên	32
1.	Mao Thị Minh Hằng	32
2.	Nguyễn Thị Thanh Hằng	32
3.	Trần Bảo Hân	32
4.	Nguyễn Thùy Trang	33
5.	Phạm Duy Trức	33
Chương 6: Kết luận		34
I.	Tóm tắt dự án	34
II.	Hạn chế	34
III.	Cải tiến trong tương lai	35

Chương 1: Giới thiệu

Dự án Facebook Clone là một hệ thống mạng xã hội mô phỏng các chức năng cốt lõi của nền tảng Facebook, được xây dựng trên nền tảng kiến trúc microservice hiện đại. Ứng dụng bao gồm các tính năng như: đăng ký/dăng nhập, quản lý bạn bè, tạo bài viết, bình luận, thích bài viết, tìm kiếm người dùng, và lịch sử tìm kiếm.

Phía backend sử dụng Spring Boot để phát triển các microservice độc lập như: post-service, user-service, message-service, search-service, và gateway-service. Dữ liệu được lưu trữ qua các hệ quản trị cơ sở dữ liệu như MySQL và PostgreSQL tùy theo đặc thù từng microservice.

Phía frontend sử dụng ReactJS để tạo giao diện hiện đại, phản hồi nhanh và tương tác tốt với người dùng.

Hệ thống cũng được triển khai bằng Docker, giúp dễ dàng mở rộng, triển khai và quản lý các container dịch vụ riêng biệt.

I. Tổng quan và mục tiêu dự án

Mục tiêu của đề tài là xây dựng một hệ thống mạng xã hội cơ bản mô phỏng các chức năng chính của Facebook, đồng thời áp dụng các công nghệ và kiến trúc phần mềm hiện đại để đảm bảo khả năng mở rộng, bảo trì và triển khai dễ dàng. Cụ thể, các mục tiêu bao gồm:

- **Xây dựng nền tảng mạng xã hội cơ bản:** Cho phép người dùng đăng ký, đăng nhập, chỉnh sửa hồ sơ cá nhân, gửi và chấp nhận lời mời kết bạn, đăng bài viết, bình luận, và tương tác với các bài viết (thích, chia sẻ, v.v.).
- **Phát triển hệ thống tìm kiếm người dùng:** Tìm kiếm theo tên hoặc email, gợi ý bạn bè, và lưu trữ lịch sử tìm kiếm của người dùng để nâng cao trải nghiệm.
- **Áp dụng kiến trúc microservices:** Thiết kế hệ thống theo hướng tách biệt các chức năng thành các dịch vụ nhỏ như `auth-service`, `user-service`, `post-service`, `search-service`, giúp dễ dàng quản lý, mở rộng và triển khai độc lập.
- **Sử dụng công nghệ hiện đại trong phát triển frontend:** Xây dựng giao diện người dùng bằng React kết hợp với Redux để quản lý trạng thái, đảm bảo giao diện thân thiện, dễ sử dụng và phản hồi nhanh.
- **Bảo mật và xác thực người dùng:** Sử dụng JSON Web Token (JWT) để xác thực và phân quyền người dùng khi truy cập hệ thống.
- **Triển khai hệ thống dễ dàng bằng Docker:** Đóng gói từng microservice dưới dạng container giúp triển khai nhanh chóng, đồng nhất trên nhiều môi trường khác nhau, hỗ trợ phát triển theo hướng DevOps.

- **Sử dụng RabbitMQ để giao tiếp bất đồng bộ giữa các service:** Tích hợp RabbitMQ để truyền thông tin giữa các microservice theo cơ chế message queue, giúp giảm độ phụ thuộc giữa các thành phần, cải thiện hiệu năng và khả năng mở rộng hệ thống.

II. Phát biểu vấn đề

Trong thời đại số hiện nay, mạng xã hội đóng vai trò quan trọng trong việc kết nối con người, chia sẻ thông tin và xây dựng cộng đồng. Các nền tảng như Facebook, Twitter, hay Instagram đã thay đổi hoàn toàn cách con người tương tác với nhau qua môi trường trực tuyến. Tuy nhiên, việc xây dựng một hệ thống mạng xã hội với đầy đủ chức năng và khả năng mở rộng cao luôn là một thách thức lớn đối với các nhà phát triển phần mềm.

Vấn đề đặt ra là: Làm thế nào để xây dựng một hệ thống mạng xã hội mô phỏng các chức năng cơ bản của Facebook, sử dụng công nghệ hiện đại, có khả năng mở rộng, dễ triển khai, bảo trì và tối ưu hiệu năng?

Cụ thể, các vấn đề chính cần giải quyết bao gồm:

- Thiết kế hệ thống theo kiến trúc microservice để đảm bảo khả năng mở rộng và tách biệt các chức năng.
- Triển khai giao diện người dùng hiện đại, dễ sử dụng và có tính tương tác cao bằng React.
- Đảm bảo tính bảo mật và xác thực người dùng bằng JWT.
- Quản lý dữ liệu người dùng, bài viết, bình luận, bạn bè, và lịch sử tìm kiếm một cách hiệu quả.
- Tích hợp các công nghệ như Docker để triển khai hệ thống một cách dễ dàng và nhất quán.

Từ những vấn đề nêu trên, nhóm thực hiện đề tài "Xây dựng hệ thống mạng xã hội Facebook Clone sử dụng Spring Boot, React và kiến trúc Microservice", nhằm mục tiêu nghiên cứu, thiết kế và phát triển một hệ thống mạng xã hội cơ bản nhưng đầy đủ chức năng, hướng tới tính ứng dụng thực tiễn trong môi trường học tập và làm việc thực tế.

III. Phạm vi dự án

Đề tài tập trung xây dựng một hệ thống mạng xã hội mô phỏng các chức năng cơ bản của nền tảng Facebook, phục vụ cho mục đích học tập và nghiên cứu công nghệ. Phạm vi của dự án bao gồm:

- **Phía người dùng (Frontend):**
 - Giao diện đăng ký, đăng nhập, chỉnh sửa hồ sơ người dùng.
 - Tìm kiếm người dùng và gợi ý bạn bè.
 - Gửi và xử lý lời mời kết bạn.
 - Đăng bài viết, xem bài viết của người khác, thích và bình luận.
 - Hiển thị lịch sử tìm kiếm gần nhất.
 - Chức năng nhắn tin (chat): giao diện gửi và nhận tin nhắn giữa các người dùng đã là bạn bè.

- **Phía hệ thống (Backend):**

- Tách biệt các chức năng thành các microservice độc lập: `auth-service`, `user-service`, `post-service`, `friend-service`, `search-service`, v.v.
- Sử dụng Spring Boot để xây dựng API cho từng service.
- Sử dụng RabbitMQ để giao tiếp bất đồng bộ giữa các microservice.
- Sử dụng JWT để xác thực người dùng.
- Quản lý dữ liệu người dùng, bài viết, bình luận, bạn bè, tin nhắn và lịch sử tìm kiếm.

- **Phía triển khai (Deployment):**

- Sử dụng Docker để đóng gói và triển khai từng service.
- Sử dụng API Gateway để định tuyến yêu cầu đến đúng microservice.

Dự án chỉ tập trung vào các chức năng cốt lõi và không bao gồm các tính năng nâng cao như gọi video, livestream, hoặc tích hợp trí tuệ nhân tạo. Ngoài ra, bảo mật nâng cao và tối ưu hiệu năng cho môi trường production chưa được triển khai đầy đủ do giới hạn thời gian và tài nguyên.

Chương 2: Cơ sở lý thuyết

I. Công nghệ và Thư viện sử dụng

Trong quá trình xây dựng hệ thống mạng xã hội Facebook Clone, nhóm đã sử dụng nhiều công nghệ và thư viện hiện đại nhằm đảm bảo hiệu năng, khả năng mở rộng và tính tương tác cao. Cụ thể bao gồm:

1. Ngôn ngữ lập trình

- **Java:** Ngôn ngữ chính được sử dụng để phát triển các microservice backend với Spring Boot.
- **TypeScript:** Sử dụng để phát triển frontend với React, giúp tăng tính an toàn và dễ bảo trì so với JavaScript thuần.

2. Framework và Thư viện Backend

- **Spring Boot:** Framework chính để xây dựng các microservice backend, cung cấp cấu hình mặc định, tích hợp dễ dàng với cơ sở dữ liệu, bảo mật và các thư viện khác.
- **Spring Security:** Dùng để xác thực và phân quyền người dùng thông qua JWT.
- **Spring Web:** Cung cấp các API REST cho các service.
- **Spring Data JPA / PostgreSQL:** Hỗ trợ thao tác với cơ sở dữ liệu quan hệ.
- **RabbitMQ (Spring AMQP):** Dùng để giao tiếp bất đồng bộ giữa các service bằng hàng đợi (queue).

3. Frontend

- **React:** Thư viện JavaScript phổ biến dùng để xây dựng giao diện người dùng hiện đại.
- **Axios:** Thư viện gửi HTTP request từ frontend đến backend.
- **Tailwind CSS / Shadcn UI:** Hỗ trợ xây dựng giao diện nhanh chóng, đẹp mắt và chuẩn responsive.

4. Cơ sở dữ liệu

- **MySQL:** Hệ quản trị cơ sở dữ liệu dùng để lưu trữ thông tin người dùng, bài viết, bạn bè, tin nhắn, v.v.

5. Giao tiếp giữa các service

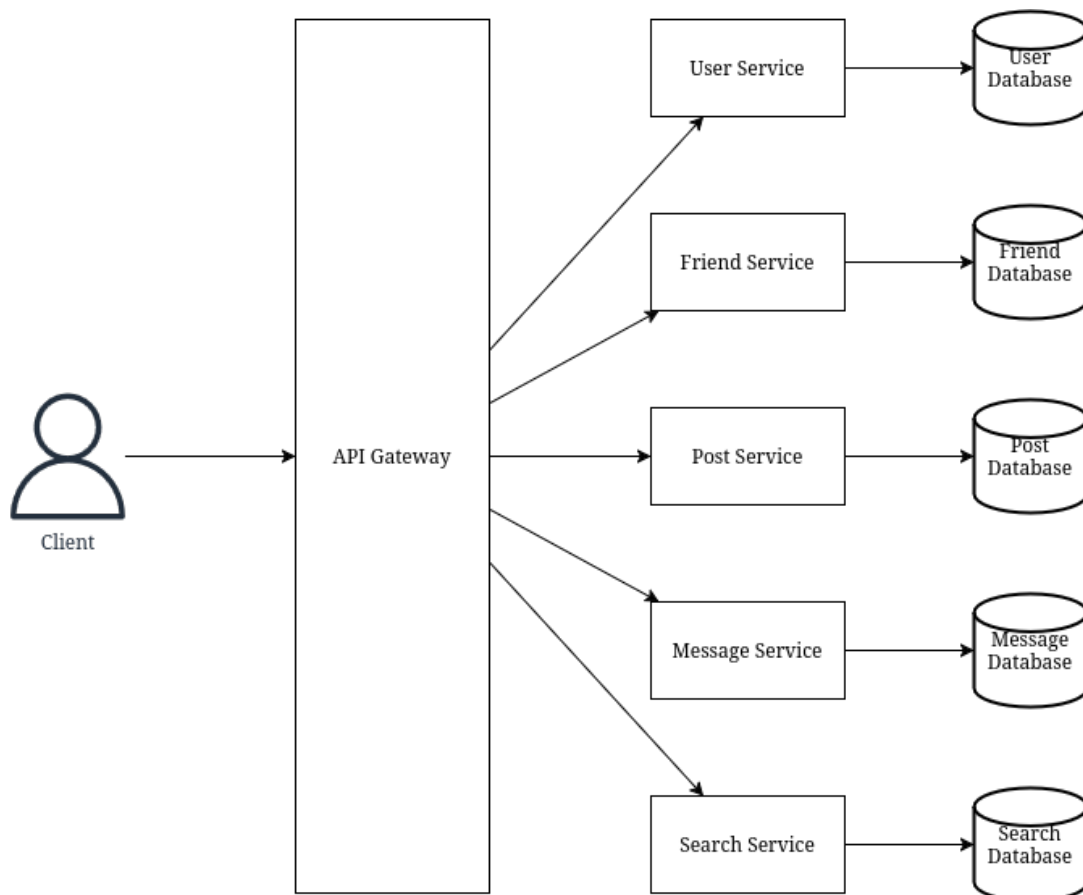
- **RabbitMQ:** Dùng để xử lý các tác vụ bất đồng bộ như gửi thông báo, tin nhắn, hoặc đồng bộ dữ liệu giữa các service.

6. Triển khai hệ thống

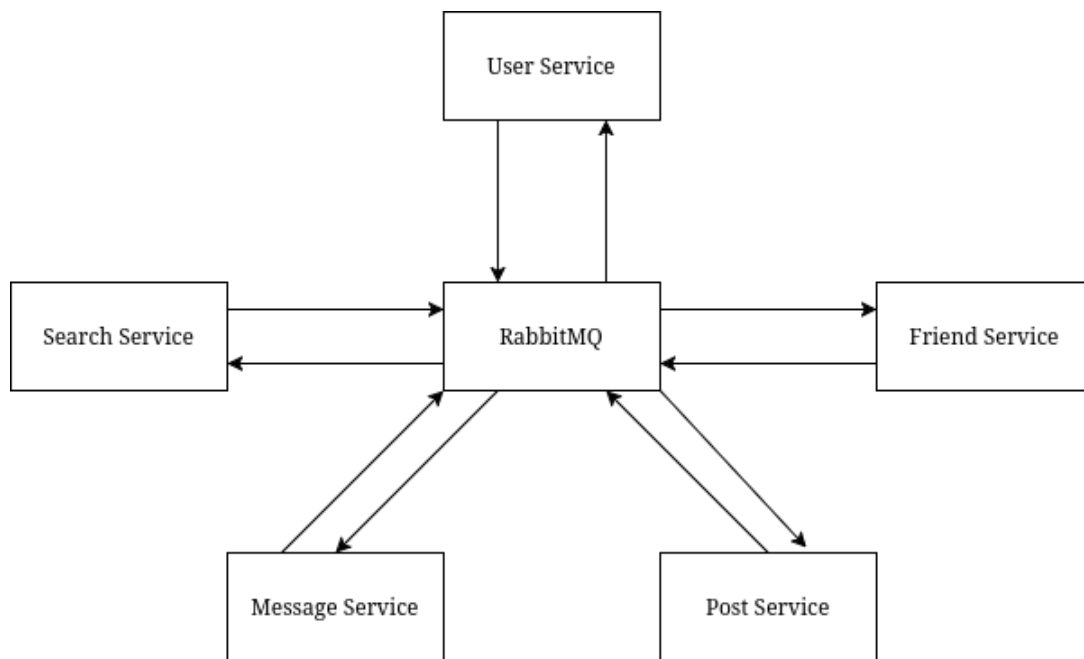
- **Docker:** Dùng để đóng gói từng microservice thành container riêng, dễ triển khai trên nhiều môi trường.
- **Docker Compose:** Quản lý và khởi chạy nhiều container cùng lúc.
- **API Gateway (Spring Cloud Gateway):** Định tuyến và kiểm soát truy cập đến các microservice từ một điểm trung tâm.

II. Kiến trúc hệ thống

Hệ thống được thiết kế theo mô hình microservices, trong đó các thành phần riêng biệt giao tiếp với nhau thông qua RabbitMQ để đảm bảo tính mở rộng, tách biệt chức năng và dễ bảo trì. Dưới đây là hai sơ đồ minh họa kiến trúc tổng thể và cơ chế trao đổi thông điệp giữa các dịch vụ.



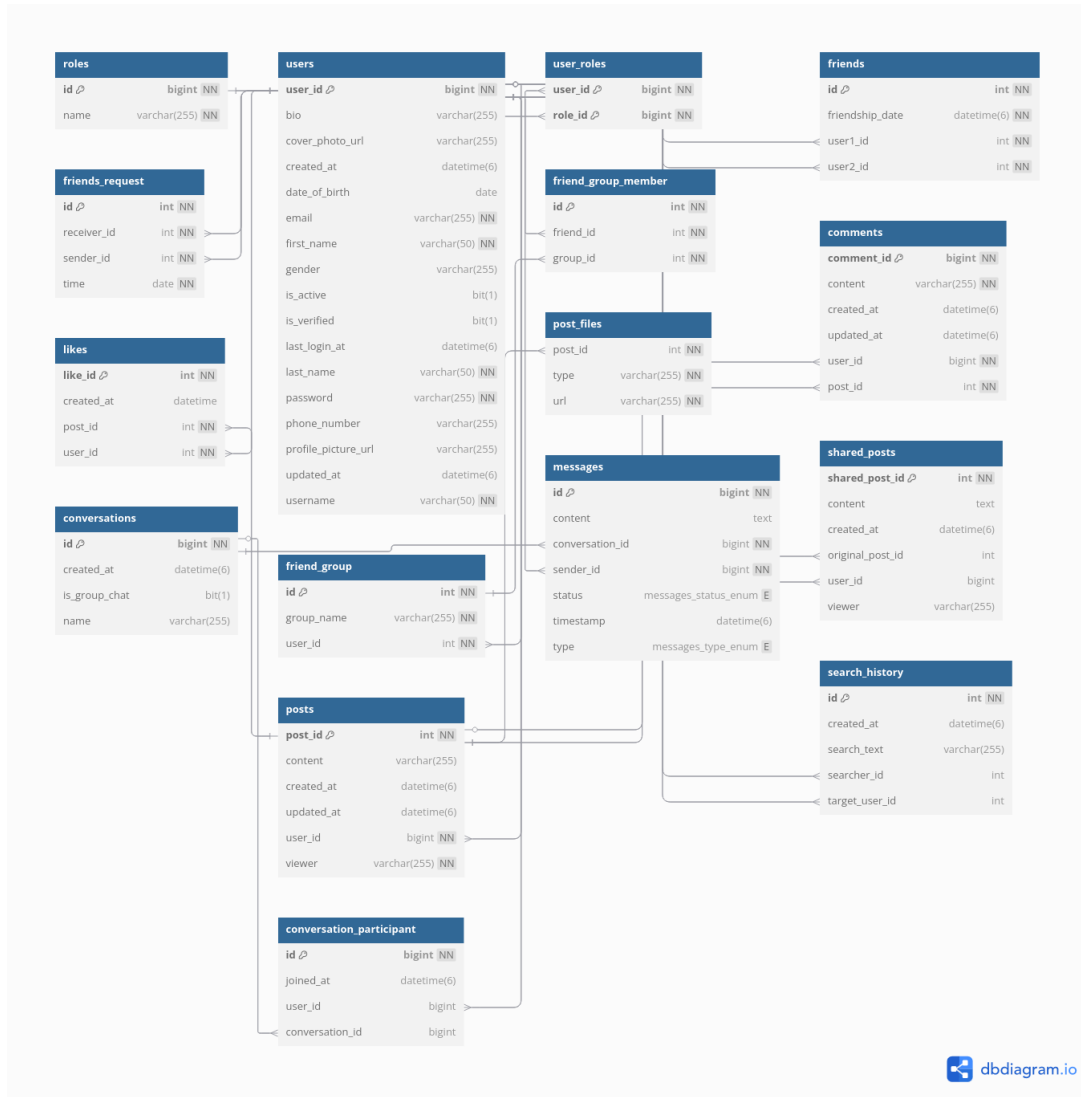
Hình 2.1: Sơ đồ kiến trúc tổng thể của hệ thống, thể hiện các thành phần chính như API Gateway, các microservice (user, message, notification, v.v.), cơ sở dữ liệu, và kênh giao tiếp nội bộ.



Hình 2.2: Minh họa cơ chế giao tiếp giữa các service thông qua RabbitMQ, nơi mỗi service có thể gửi và nhận thông điệp không đồng bộ thông qua hàng đợi.

III. Thiết kế cơ sở dữ liệu

1. Sơ đồ cơ sở dữ liệu (sơ đồ ER)



Hình 2.3: Sơ đồ ER của hệ thống

2. Các bảng và mối quan hệ

Cơ sở dữ liệu hệ thống được thiết kế với nhiều bảng để quản lý người dùng, vai trò, bài viết, bình luận, bạn bè, tin nhắn, lịch sử tìm kiếm và các hoạt động tương tác. Dưới đây là một số bảng tiêu biểu:

- **users**: Lưu thông tin chi tiết của người dùng bao gồm tên, email, mật khẩu, ảnh đại diện, ảnh bìa, giới tính, ngày sinh, trạng thái xác thực, thời gian đăng ký và cập nhật.
- **roles**, **user_roles**: Quản lý vai trò của người dùng trong hệ thống với quan hệ nhiều-nhiều giữa bảng **users** và **roles**.
- **friends**: Lưu mối quan hệ bạn bè giữa hai người dùng đã chấp nhận kết bạn, bao gồm ngày trở thành bạn bè.

- **friends_request**: Quản lý các yêu cầu kết bạn, lưu người gửi, người nhận và thời gian gửi yêu cầu.
- **friend_group, friend_group_member**: Cho phép người dùng tạo nhóm bạn và thêm thành viên vào nhóm.
- **posts**: Quản lý bài viết của người dùng bao gồm nội dung, ngày đăng, trạng thái cập nhật và mức độ riêng tư (**viewer**).
- **post_files**: Lưu các file đa phương tiện đi kèm bài viết như ảnh hoặc video.
- **shared_posts**: Quản lý các bài viết được chia sẻ lại từ người dùng khác, có thể kèm nội dung bình luận và mức độ riêng tư.
- **comments**: Quản lý bình luận của người dùng trên bài viết.
- **likes**: Lưu thông tin người dùng thích bài viết.
- **conversations, conversation_participant, messages**: Quản lý hệ thống nhắn tin giữa người dùng, hỗ trợ hội thoại nhóm, lưu tin nhắn kèm loại và trạng thái.
- **search_history**: Ghi lại lịch sử tìm kiếm người dùng khác trong hệ thống.

3. Phương pháp mô hình hóa dữ liệu

Để đảm bảo dữ liệu được tổ chức khoa học, dễ mở rộng và dễ bảo trì, nhóm đã áp dụng phương pháp mô hình hóa dữ liệu như sau:

- **Mô hình quan hệ (Relational Model)**: Được sử dụng trong các microservice có tính nhất quán cao như quản lý người dùng, bài viết, bình luận,... Các bảng được chuẩn hóa theo các chuẩn 1NF, 2NF và 3NF để tránh dư thừa dữ liệu và đảm bảo toàn vẹn.
- **Phân tách theo microservice**: Mỗi service có cơ sở dữ liệu riêng (Database-per-Service), đảm bảo tính độc lập trong triển khai và dễ dàng mở rộng từng phần riêng biệt. Dữ liệu được đồng bộ hoặc giao tiếp giữa các service thông qua RabbitMQ hoặc REST API.
- **Mô hình hóa thực thể - quan hệ (ERD)**: Nhóm đã xây dựng sơ đồ ER để mô tả các thực thể chính (User, Post, Comment, Friend, Message, v.v.) và các mối quan hệ giữa chúng (1-1, 1-n, n-n), làm cơ sở cho thiết kế cơ sở dữ liệu quan hệ.
- **Quy ước đặt tên và kiểu dữ liệu**: Các bảng, cột và kiểu dữ liệu được đặt tên nhất quán theo tiếng Anh, có định nghĩa rõ ràng về khóa chính, khóa ngoại và chỉ mục để tối ưu truy vấn.

Chương 3: Hiện thực

I. Triển khai các tính năng

1. Message Service

- **Gửi tin nhắn:**
 - Cho phép người dùng gửi tin nhắn văn bản hoặc media.
 - Tự động lưu thời gian gửi và thông tin người gửi/người nhận.
- **Xem tin nhắn:**
 - Lấy toàn bộ tin nhắn giữa hai người dùng cụ thể, sắp xếp theo thời gian gửi.
- **Xóa tin nhắn:**
 - Cho phép người dùng xóa tin nhắn theo ID.
 - Chỉ cho phép xóa nếu người dùng là người gửi.
- **Kiểm tra quyền:**
 - Tự động xác định người dùng từ JWT trong cookie gửi kèm request.
 - Chỉ cho phép thao tác nếu người dùng liên quan đến cuộc trò chuyện.

2. Friend Service

- **Gửi lời mời kết bạn:**
 - Kiểm tra sự tồn tại của người dùng được mời.
 - Kiểm tra trạng thái lời mời trước đó (đã gửi, đã là bạn, v.v.).
 - Tạo bản ghi song song (user1 ↔ user2).
- **Xóa bạn:**
 - Xóa cả hai bản ghi bạn bè (hai chiều).
- **Kiểm tra trạng thái bạn bè:**
 - Kiểm tra xem hai user có phải là bạn không (theo cả hai chiều).
- **Xem danh sách bạn bè:**
 - Lấy danh sách tất cả user mà người dùng hiện tại là bạn.
 - Gọi API ‘UserService’ để lấy thông tin chi tiết của từng bạn bè.

3. Search History Service

- **Lưu lịch sử tìm kiếm:**
 - Ghi lại thông tin tìm kiếm bao gồm: người tìm kiếm, người bị tìm, văn bản tìm kiếm, thời gian tìm.
 - Nếu đã tồn tại bản ghi tương tự (trùng người tìm, người bị tìm và văn bản), thì xóa bản cũ và lưu bản mới.
- **Xem lịch sử tìm kiếm gần đây:**
 - Trả về danh sách các kết quả tìm kiếm gần đây của người dùng, sắp xếp theo thời gian giảm dần.
 - Mỗi bản ghi đi kèm thông tin chi tiết về người bị tìm từ ‘UserService’.
- **Xóa một bản ghi lịch sử:**
 - Cho phép người dùng xóa một bản ghi cụ thể theo ID.
 - Kiểm tra người dùng hiện tại có quyền xóa bản ghi hay không.
- **Xóa lịch sử theo ID người dùng:**
 - Cho phép xóa bản ghi tìm kiếm theo ‘ID’ người dùng.
 - Phục vụ mục đích dọn dẹp dữ liệu khi người dùng xóa tài khoản.
- **Xóa toàn bộ lịch sử tìm kiếm:**
 - Cho phép xóa toàn bộ lịch sử tìm kiếm trong hệ thống.
 - Thường dùng bởi admin hoặc trong quá trình reset dữ liệu.

4. User Service

- **Lấy thông tin người dùng theo ID:**
 - Truy vấn thông tin chi tiết của người dùng dựa trên ID.
 - Nếu không tìm thấy người dùng, trả về lỗi.
- **Lấy thông tin người dùng từ JWT Token:**
 - Phân tích và xác thực token để trích xuất username.
 - Trả về thông tin người dùng tương ứng với username từ token.
 - Kiểm tra chữ ký, tính hợp lệ, và hạn sử dụng của JWT.
- **Chuyển đổi đối tượng User sang DTO:**
 - Lấy các trường cần thiết từ đối tượng ‘User’ để đưa vào ‘UserDTO’.
 - Bao gồm thông tin cá nhân, ảnh đại diện, vai trò, trạng thái hoạt động, thời gian tạo/cập nhật...

5. Post Service

– Đăng bài viết:

- Các bước để đăng bài viết vào hệ thống.
- Kiểm tra các trường thông tin, xác thực và lưu vào cơ sở dữ liệu.

– Đăng bài viết:

- Cho phép người dùng đăng bài viết với nội dung văn bản và/hoặc hình ảnh, video.
- Lưu thông tin thời gian đăng, người đăng, trạng thái công khai/bạn bè/riêng tư.

– Tương tác với bài viết (Like):

- Cho phép người dùng thích hoặc bỏ thích bài viết.
- Mỗi người chỉ được like một lần mỗi bài viết.

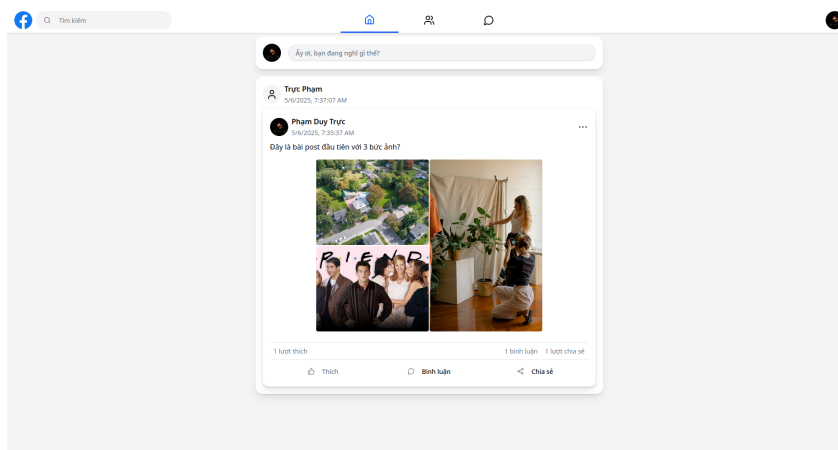
– Chia sẻ bài viết (Share):

- Người dùng có thể chia sẻ bài viết của người khác lên trang cá nhân của mình.
- Ghi nhận người chia sẻ và bài viết gốc được chia sẻ.

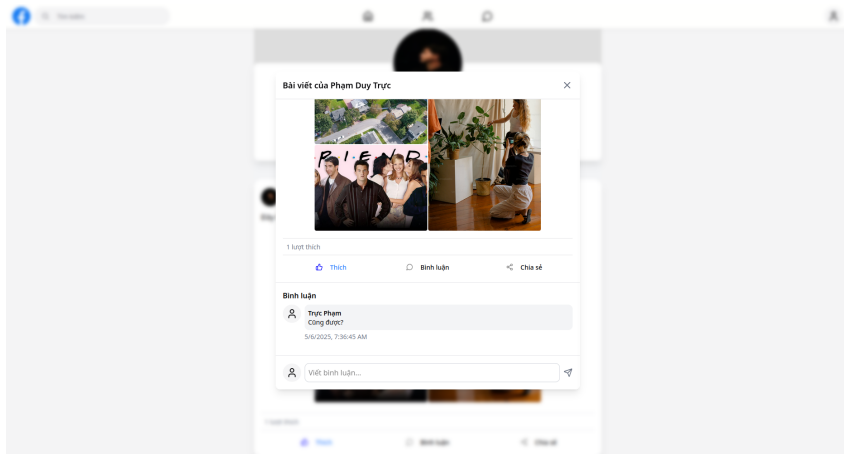
– Bình luận bài viết (Comment):

- Cho phép người dùng bình luận bài viết bằng văn bản hoặc emoji.
- Hỗ trợ phân cấp bình luận (trả lời bình luận).
- Lưu thời gian tạo và người bình luận.

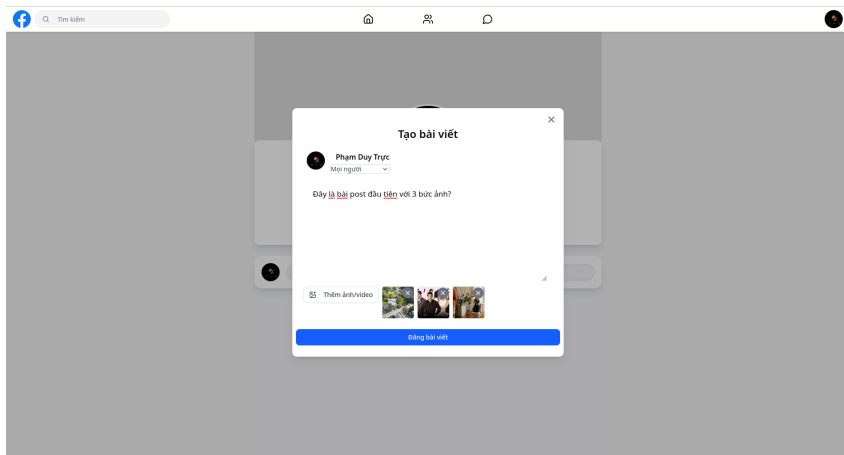
II. Ảnh chụp màn hình và minh họa



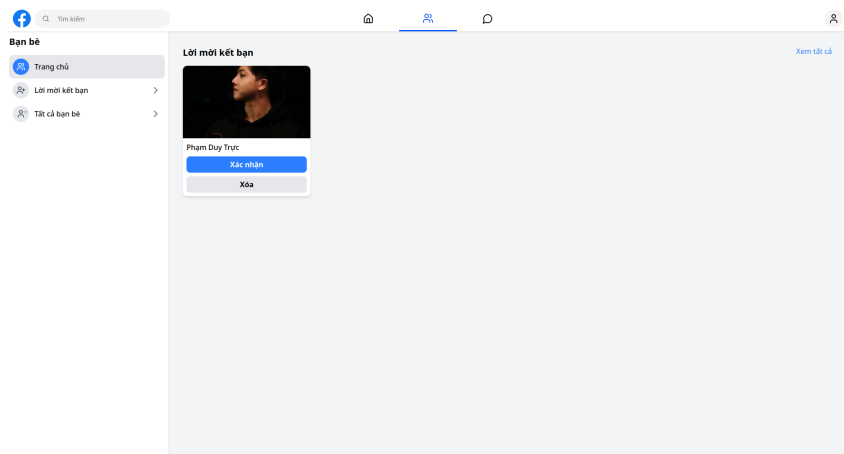
Hình 3.1: Trang chủ với bài viết được chia sẻ



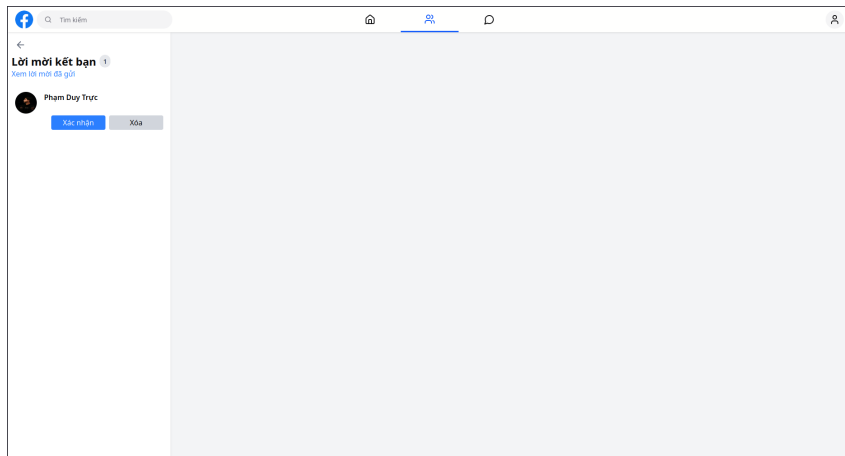
Hình 3.2: Hộp thoại bình luận



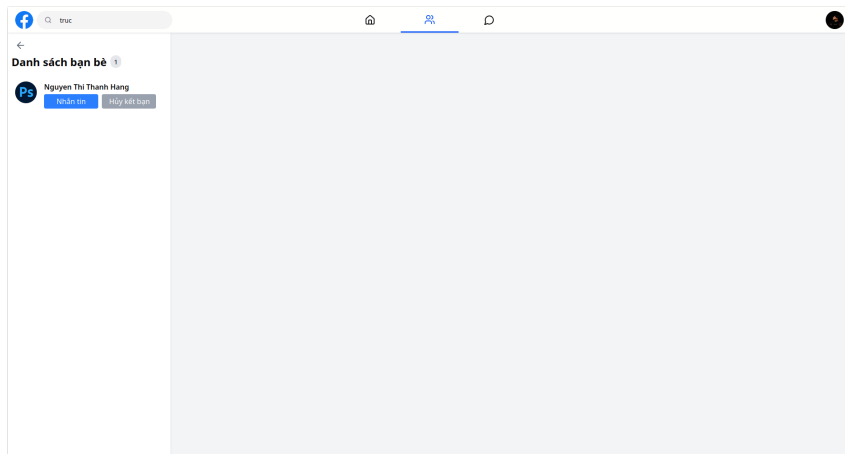
Hình 3.3: Giao diện tạo bài viết mới



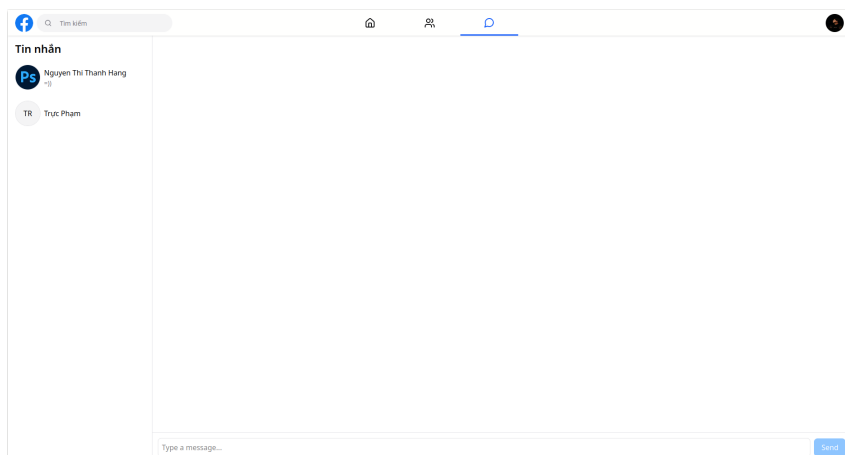
Hình 3.4: Trang bạn bè



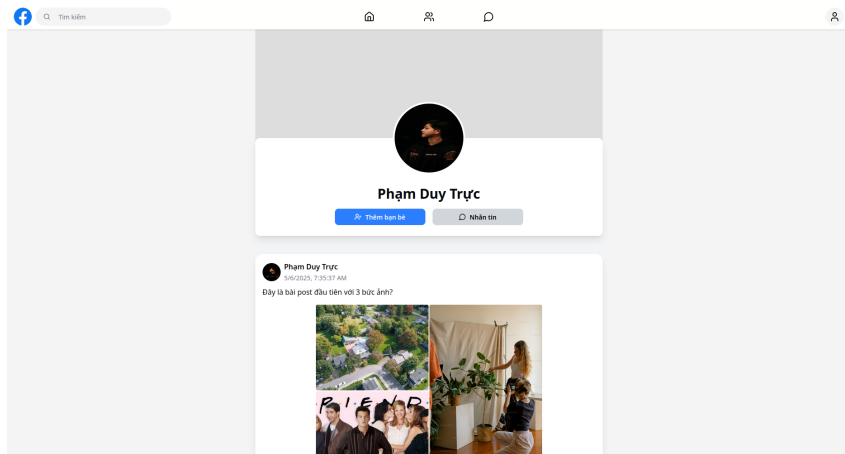
Hình 3.5: Danh sách lời mời kết bạn



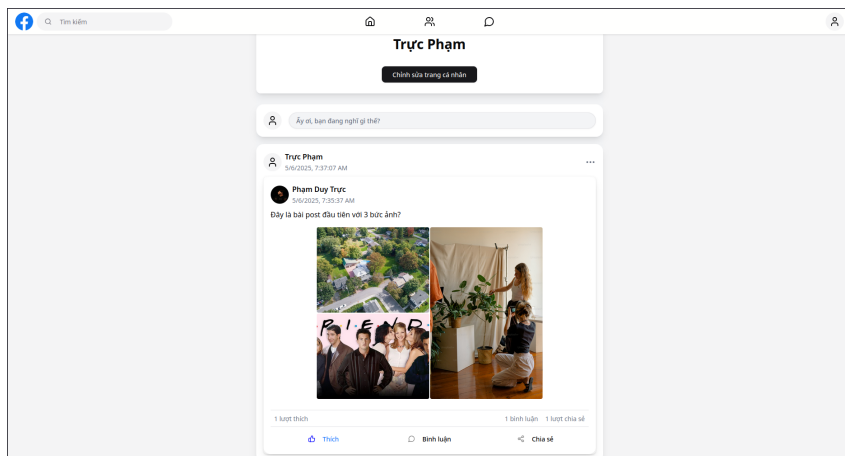
Hình 3.6: Danh sách bạn bè



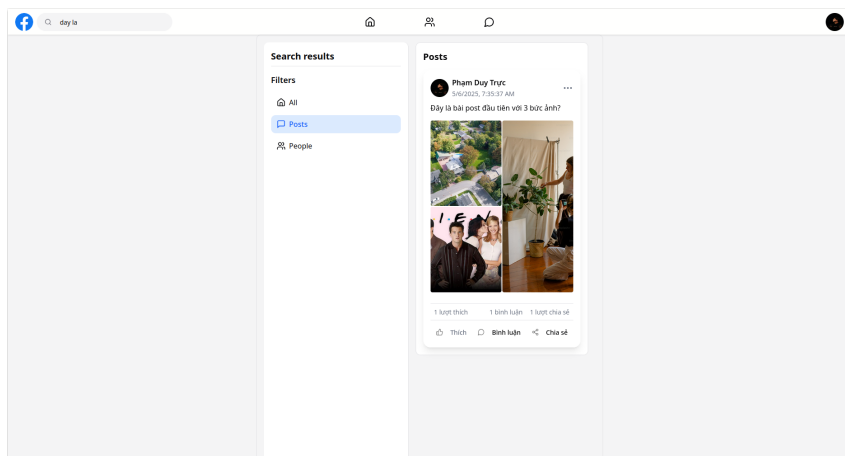
Hình 3.7: Giao diện trò chuyện tin nhắn



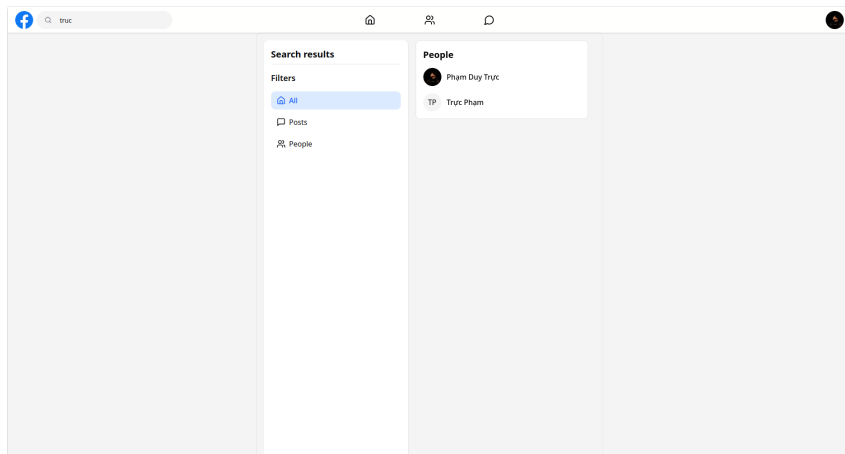
Hình 3.8: Trang cá nhân người dùng



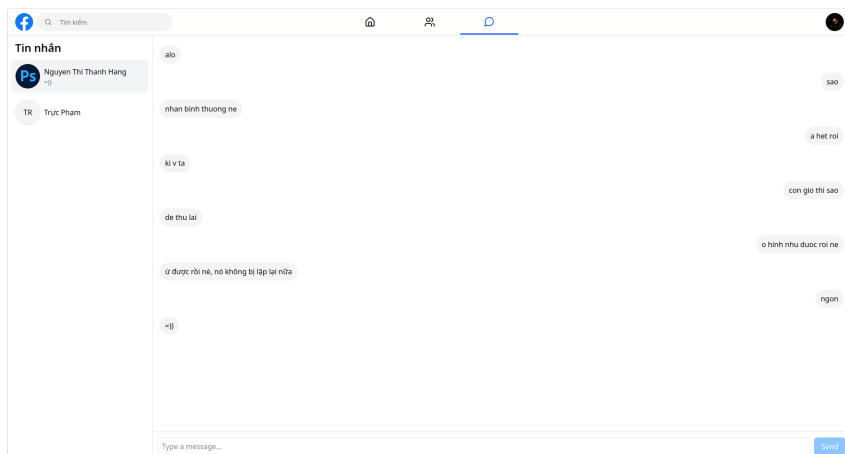
Hình 3.9: Trang cá nhân với bài viết



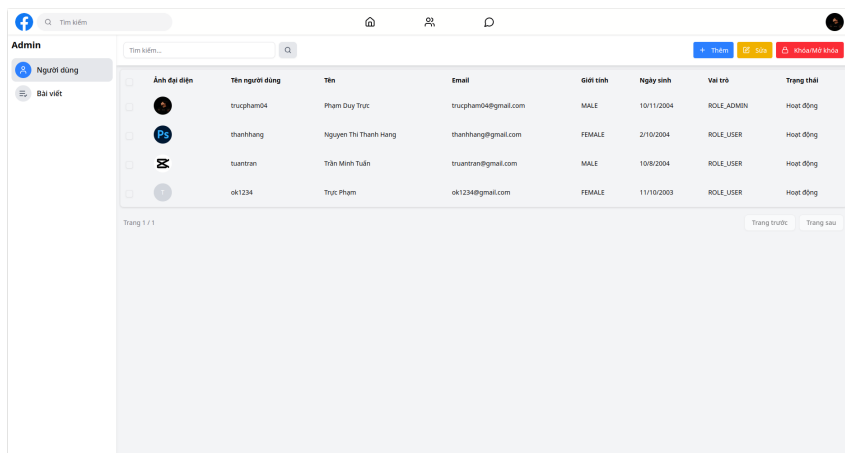
Hình 3.10: Kết quả tìm kiếm bài viết



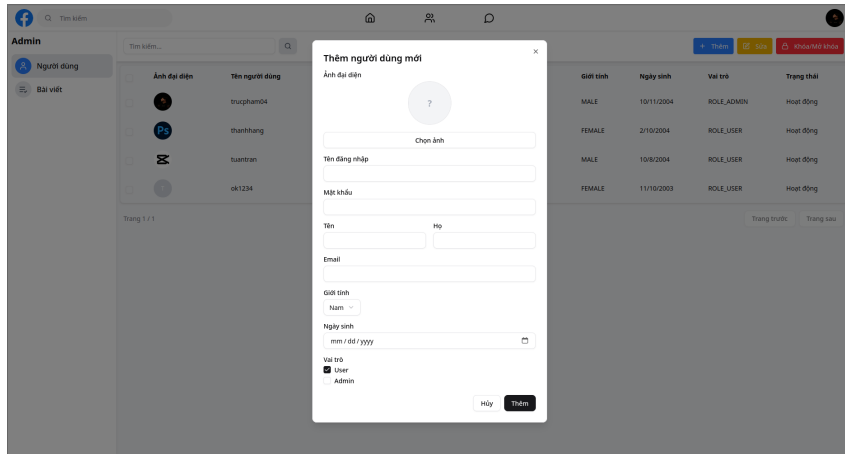
Hình 3.11: Kết quả tìm kiếm người dùng



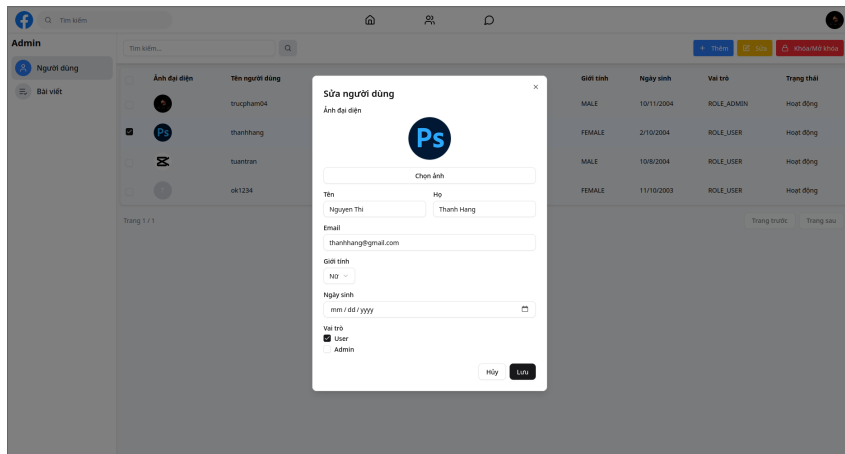
Hình 3.12: Chi tiết cuộc trò chuyện



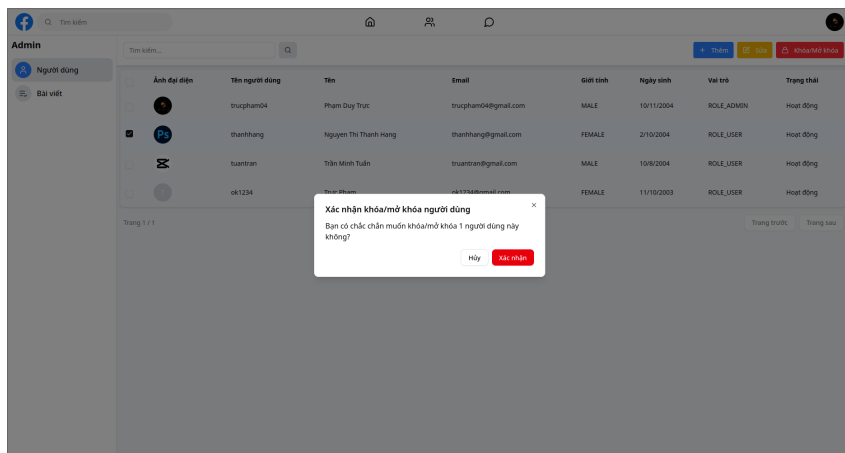
Hình 3.13: Trang quản lý người dùng của quản trị viên



Hình 3.14: Quản trị viên tạo người dùng mới



Hình 3.15: Quản trị viên cập nhật thông tin người dùng



Hình 3.16: Quản trị viên khóa tài khoản người dùng

III. Cấu trúc mã nguồn

1. Tổ chức thư mục

Cấu trúc thư mục của dự án được tổ chức như sau:

- **Thư mục dịch vụ:**
 - **api_gateway/**: Dịch vụ cổng API, quản lý các yêu cầu đến các dịch vụ khác.
 - **friend_service/**: Dịch vụ quản lý thông tin bạn bè.
 - **frontend/**: Module giao diện người dùng.
 - **message_service/**: Dịch vụ xử lý tin nhắn.
 - **post_service/**: Dịch vụ xử lý bài đăng.
 - **search_service/**: Dịch vụ tìm kiếm.
 - **user_service/**: Dịch vụ quản lý thông tin người dùng.
- **Tệp cấu hình:**
 - **.gitignore**: Tệp cấu hình để bỏ qua các tệp/thư mục không muốn đưa vào hệ thống quản lý phiên bản Git.
 - **docker-compose.yml**: Tệp cấu hình Docker Compose để quản lý các dịch vụ container hóa.
 - **package-lock.json**: Tệp khóa phiên bản các phụ thuộc, đảm bảo tính nhất quán khi cài đặt.
 - **package.json**: Tệp cấu hình chính của dự án, chứa thông tin phụ thuộc, script và metadata.
 - **README.md**: Tài liệu hướng dẫn, cung cấp thông tin tổng quan và cách thiết lập dự án.

Cấu trúc này hỗ trợ việc triển khai microservices và quản lý phụ thuộc một cách hiệu quả.

1.1 Cấu trúc thư mục frontend

Cấu trúc thư mục của **frontend** được tổ chức như sau:

- **node_modules/**: Chứa các thư viện và phụ thuộc của dự án, được quản lý bởi npm hoặc yarn.
- **public/**: Chứa các tệp tĩnh được phục vụ trực tiếp, như favicon, manifest, hoặc các tệp HTML cơ bản.
- **src/**: Thư mục chính chứa mã nguồn của giao diện người dùng.
 - **assets/**: Chứa các tài nguyên tĩnh như hình ảnh, biểu tượng, hoặc phông chữ.
 - **components/**: Các thành phần giao diện người dùng tái sử dụng.
 - * **app/**: Các thành phần liên quan đến ứng dụng chính.
 - * **layouts/**: Các thành phần bố cục chung (layout).
 - * **message/**: Thành phần giao diện cho chức năng tin nhắn.

- * **post**/: Thành phần giao diện cho bài đăng.
- * **search**/: Thành phần giao diện cho chức năng tìm kiếm.
- * **ui**/: Các thành phần giao diện chung (UI components).
- **context**/: Chứa các context để quản lý trạng thái toàn cục.
- **hooks**/: Các custom hooks để tái sử dụng logic trong React.
- **lib**/: Các thư viện hoặc module tiện ích.
- **pages**/: Các trang chính của ứng dụng.
 - * **auth**/: Trang liên quan đến xác thực (đăng nhập, đăng ký).
 - * **main**/: Trang chính của ứng dụng.
 - * **profile**/: Trang hồ sơ người dùng.
- **routes**/: Quản lý định tuyến trong ứng dụng.
- **services**/: Các dịch vụ gọi API hoặc xử lý logic nghiệp vụ.
- **styles**/: Các tệp CSS hoặc SCSS để định dạng giao diện.
- **types**/: Định nghĩa các kiểu dữ liệu (TypeScript).
- **utils**/: Các hàm tiện ích hỗ trợ.
- **App.tsx**: Tệp chính của ứng dụng, khởi tạo giao diện và logic chính.

Cấu trúc này đảm bảo tính mô-đun và dễ bảo trì cho giao diện người dùng của dự án.

1.2 Cấu trúc thư mục friend_service

Cấu trúc thư mục của **friend_service** được tổ chức như sau:

- **.mvn**/: Thư mục chứa các tệp cấu hình cho Maven Wrapper, hỗ trợ quản lý phụ thuộc và xây dựng dự án.
- **src**/: Thư mục chính chứa mã nguồn của dịch vụ.
 - **main**/: Chứa mã nguồn chính của dịch vụ.
 - * **java/com/example/friend**/: Gói chính chứa mã nguồn Java.
 - **Config**/: Chứa các lớp cấu hình cho dịch vụ.
 - **controller**/: Chứa các bộ điều khiển (controllers) xử lý yêu cầu HTTP.
 - **DTO**/: Chứa các đối tượng truyền dữ liệu (Data Transfer Objects).
 - **Entity**/: Chứa các thực thể (entities) ánh xạ với cơ sở dữ liệu.
 - **Repository**/: Chứa các lớp repository để truy cập dữ liệu.
 - **service**/: Chứa các lớp dịch vụ (services) xử lý logic nghiệp vụ.
 - **FriendServiceApplication.java**: Tệp chính khởi chạy ứng dụng Spring Boot.
 - * **resources**/: Chứa các tài nguyên tĩnh và tệp cấu hình (ví dụ: `application.properties`).
 - **test**/: Chứa mã nguồn kiểm thử (unit tests, integration tests).
- **target**/: Thư mục chứa các tệp đầu ra sau khi biên dịch và xây dựng dự án (ví dụ: tệp JAR).

Cấu trúc này tuân theo chuẩn của một dự án Spring Boot, đảm bảo tính tổ chức và dễ bảo trì.

1.3 Cấu trúc thư mục `post_service`

Cấu trúc thư mục của `post_service` được tổ chức như sau:

- **.mvn/**: Thư mục chứa các tệp cấu hình cho Maven Wrapper, hỗ trợ quản lý phụ thuộc và xây dựng dự án.
- **src/**: Thư mục chính chứa mã nguồn của dịch vụ.
 - **main/**: Chứa mã nguồn chính của dịch vụ.
 - * **java/com/xdpmtmhpl/post/**: Gói chính chứa mã nguồn Java.
 - **client/**: Chứa các lớp client để giao tiếp với các dịch vụ khác.
 - **controller/**: Chứa các bộ điều khiển (controllers) xử lý yêu cầu HTTP.
 - **dto/**: Chứa các đối tượng truyền dữ liệu (Data Transfer Objects).
 - **models/**: Chứa các thực thể (entities) ánh xạ với cơ sở dữ liệu.
 - **payload/**: Chứa các lớp định nghĩa payload cho các yêu cầu hoặc phản hồi.
 - **repository/**: Chứa các lớp repository để truy cập dữ liệu.
 - **services/**: Chứa các lớp dịch vụ (services) xử lý logic nghiệp vụ.
 - **PostServiceApplication.java**: Tệp chính khởi chạy ứng dụng Spring Boot.
 - * **resources/**: Chứa các tài nguyên tĩnh và tệp cấu hình (ví dụ: `application.properties`).
 - **test/**: Chứa mã nguồn kiểm thử (unit tests, integration tests).
- **target/**: Thư mục chứa các tệp đầu ra sau khi biên dịch và xây dựng dự án (ví dụ: tệp JAR).

Cấu trúc này tuân theo chuẩn của một dự án Spring Boot, hỗ trợ quản lý bài đăng hiệu quả.

1.4 Cấu trúc thư mục `search_service`

Cấu trúc thư mục của `search_service` được tổ chức như sau:

- **.idea/**: Thư mục chứa các tệp cấu hình của IntelliJ IDEA, hỗ trợ phát triển dự án.
- **.mvn/**: Thư mục chứa các tệp cấu hình cho Maven Wrapper, hỗ trợ quản lý phụ thuộc và xây dựng dự án.
- **src/**: Thư mục chính chứa mã nguồn của dịch vụ.
 - **main/**: Chứa mã nguồn chính của dịch vụ.
 - * **java/com/example/search/**: Gói chính chứa mã nguồn Java.
 - **config/**: Chứa các lớp cấu hình cho dịch vụ.
 - **controller/**: Chứa các bộ điều khiển (controllers) xử lý yêu cầu HTTP.
 - **model/**: Chứa các thực thể (entities) ánh xạ với cơ sở dữ liệu.
 - **repository/**: Chứa các lớp repository để truy cập dữ liệu.
 - **service/**: Chứa các lớp dịch vụ (services) xử lý logic nghiệp vụ.

- **SearchbackendApplication.java**: Tập chính khởi chạy ứng dụng Spring Boot.
- **ServeltInitializer.java**: Tập khởi tạo Servlet, hỗ trợ triển khai ứng dụng trên máy chủ.
- * **resources/**: Chứa các tài nguyên tĩnh và tệp cấu hình (ví dụ: `application.properties`)
- **test/**: Chứa mã nguồn kiểm thử (unit tests, integration tests).

Cấu trúc này tuân theo chuẩn của một dự án Spring Boot, hỗ trợ chức năng tìm kiếm hiệu quả.

1.5 Cấu trúc thư mục user_service

Cấu trúc thư mục của `user_service` được tổ chức như sau:

- **.mvn/**: Thư mục chứa các tệp cấu hình cho Maven Wrapper, hỗ trợ quản lý phụ thuộc và xây dựng dự án.
- **src/**: Thư mục chính chứa mã nguồn của dịch vụ.
 - **main/**: Chứa mã nguồn chính của dịch vụ.
 - * **java/com/xdpmtmhpl/user/**: Gói chính chứa mã nguồn Java.
 - **config/**: Chứa các lớp cấu hình cho dịch vụ.
 - **controller/**: Chứa các bộ điều khiển (controllers) xử lý yêu cầu HTTP.
 - **dto/**: Chứa các đối tượng truyền dữ liệu (Data Transfer Objects).
 - **models/**: Chứa các thực thể (entities) ánh xạ với cơ sở dữ liệu.
 - **payload/**: Chứa các lớp định nghĩa payload cho các yêu cầu hoặc phản hồi.
 - **repository/**: Chứa các lớp repository để truy cập dữ liệu.
 - **security/**: Chứa các lớp xử lý bảo mật (ví dụ: xác thực, phân quyền).
 - **UserServiceApplication.java**: Tập chính khởi chạy ứng dụng Spring Boot.
 - * **resources/**: Chứa các tài nguyên tĩnh và tệp cấu hình (ví dụ: `application.properties`)
 - **test/**: Chứa mã nguồn kiểm thử (unit tests, integration tests).

Cấu trúc này tuân theo chuẩn của một dự án Spring Boot, hỗ trợ quản lý thông tin người dùng hiệu quả.

2. Các thành phần chính và tương tác

Dự án được xây dựng dựa trên kiến trúc microservices, với các thành phần chính tương tác thông qua API Gateway và giao tiếp không đồng bộ khi cần thiết. Các thành phần chính bao gồm:

- **API Gateway**: Đóng vai trò là điểm vào duy nhất, định tuyến các yêu cầu từ client đến các dịch vụ phù hợp (`user_service`, `post_service`, `notification_service`, v.v.). API Gateway cũng xử lý xác thực và phân quyền cơ bản.
- **Frontend**: Giao diện người dùng, được phát triển bằng React và TypeScript. Frontend gửi yêu cầu đến API Gateway để tương tác với các dịch vụ backend, ví dụ: lấy danh sách bài đăng từ `post_service` hoặc thông tin người dùng từ `user_service`.

- **User Service:** Quản lý thông tin người dùng, bao gồm đăng ký, đăng nhập và cập nhật hồ sơ. Dịch vụ này cung cấp API cho các dịch vụ khác (như `friend_service`) để truy vấn thông tin người dùng.
- **Friend Service:** Xử lý các chức năng liên quan đến bạn bè, như thêm bạn, xóa bạn, và lấy danh sách bạn bè. Dịch vụ này gọi `user_service` để lấy thông tin người dùng và lưu trữ dữ liệu quan hệ bạn bè.
- **Post Service:** Quản lý bài đăng, bao gồm tạo, chỉnh sửa, xóa và lấy danh sách bài đăng. Dịch vụ này tương tác với `user_service` để xác minh người dùng và với `notification_service` để gửi thông báo khi có bài đăng mới.
- **Search Service:** Cung cấp chức năng tìm kiếm bài đăng và người dùng. Dịch vụ này tương tác với `post_service` và `user_service` để truy xuất dữ liệu, sau đó xử lý và trả về kết quả tìm kiếm.
- **Message Service:** Quản lý tin nhắn giữa các người dùng, bao gồm gửi, nhận và lưu trữ tin nhắn. Dịch vụ này gọi `user_service` để xác minh người dùng và `notification_service` để gửi thông báo khi có tin nhắn mới.

Tương tác giữa các thành phần:

- Các dịch vụ giao tiếp với nhau thông qua RabbitMQ.
- API Gateway đảm bảo các yêu cầu được định tuyến chính xác và giảm tải cho các dịch vụ backend.
- Frontend hiển thị dữ liệu bằng cách gọi API từ API Gateway, đồng thời xử lý các tương tác người dùng như gửi tin nhắn hoặc tìm kiếm bài đăng.

Cấu trúc này đảm bảo tính độc lập giữa các dịch vụ, dễ dàng mở rộng và bảo trì.

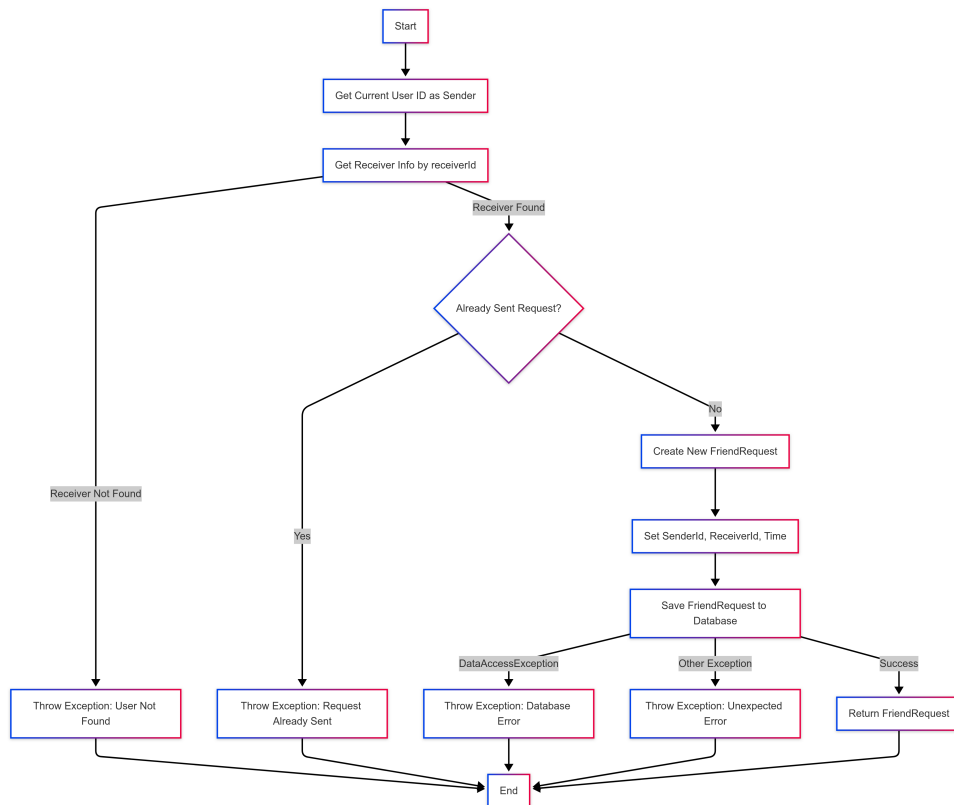
3. Flowchart cho các quy trình phức tạp

Trong phần này, chúng tôi trình bày các biểu đồ luồng (flowchart) minh họa các quy trình phức tạp trong hệ thống, bao gồm các quy trình như gửi yêu cầu kết bạn, xác thực token, gửi tin nhắn, tạo cuộc hội thoại, và thêm bạn bè. Mỗi biểu đồ luồng được tạo bằng công cụ Mermaid và lưu dưới dạng hình ảnh PNG. Dưới đây là các biểu đồ cùng với giải thích chi tiết.

3.1 Gửi yêu cầu kết bạn (Send Request)

Giải thích quy trình: Quy trình này được thực hiện trong `FriendRequestService.java` (phương thức `SendRequest`). Biểu đồ luồng minh họa các bước để gửi yêu cầu kết bạn từ người dùng hiện tại (`senderId`) đến người nhận (`receiverId`).

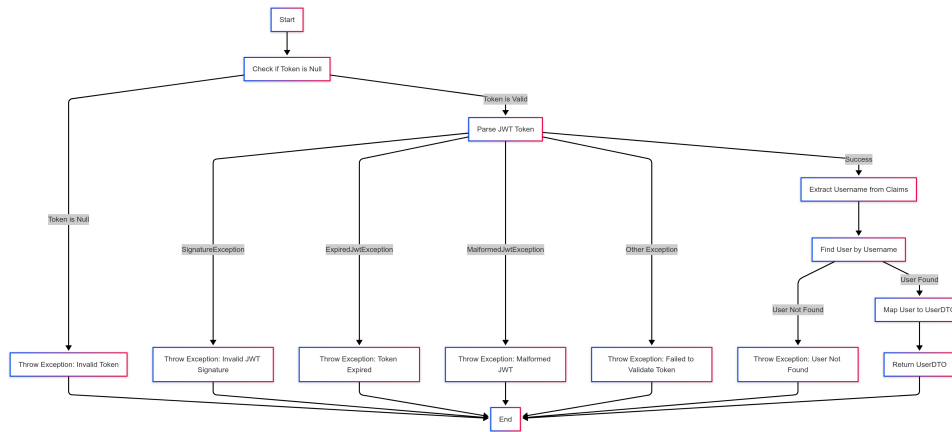
- **Bắt đầu:** Quy trình bắt đầu khi nhận yêu cầu gửi lời mời kết bạn với `receiverId`.
- **Lấy ID người gửi:** Sử dụng `getCurrentUserId()` để lấy `senderId` từ token JWT.
- **Lấy thông tin người nhận:** Gọi `userClient.getUserById(receiverId)` để kiểm tra xem người nhận có tồn tại không.
 - Nếu **không tồn tại**, ném ngoại lệ "User with ID `receiverId` does not exist" và kết thúc.



Hình 3.17: Biểu đồ luồng cho quy trình gửi yêu cầu kết bạn.

- Nếu **tồn tại**, tiếp tục.
- **Kiểm tra đã gửi yêu cầu chưa:** Gọi `friendRequestRepository.existsBySenderIdAndReceiverId(receiverId)` để kiểm tra xem đã có yêu cầu kết bạn chưa.
 - Nếu **đã gửi**, ném ngoại lệ "You are request friends with user `receiverId`" và kết thúc.
 - Nếu **chưa gửi**, tiếp tục.
- **Tạo yêu cầu kết bạn:** Tạo một đối tượng `FriendRequest` mới.
- **Thiết lập thông tin yêu cầu:** Gán `senderId`, `receiverId`, và thời gian (`LocalDate.now()`).
- **Lưu yêu cầu:** Gọi `friendRequestRepository.save` để lưu yêu cầu vào cơ sở dữ liệu.
 - Nếu **thành công**, trả về `FriendRequest`.
 - Nếu **`DataAccessException`**, ném ngoại lệ "Database error".
 - Nếu **ngoại lệ khác**, ném ngoại lệ "Unexpected error".
- **Kết thúc:** Kết thúc quy trình.

Quy trình này đảm bảo rằng yêu cầu kết bạn chỉ được gửi khi người nhận tồn tại và chưa có yêu cầu nào được gửi trước đó, đồng thời xử lý các trường hợp lỗi như lỗi cơ sở dữ liệu.



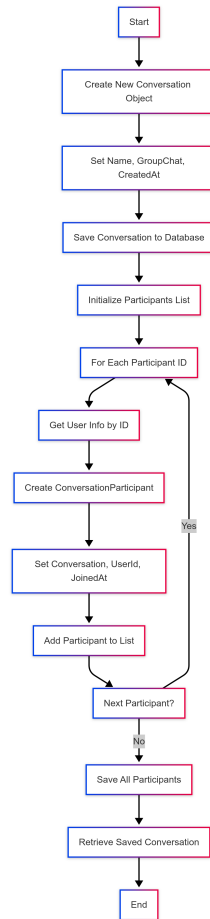
Hình 3.18: Biểu đồ luồng cho quy trình xác thực token và lấy thông tin người dùng.

3.2 Xác thực token và lấy thông tin người dùng (Get User by Token)

Giải thích quy trình: Quy trình này được thực hiện trong `UserService.java` (phương thức `getUserByToken`). Biểu đồ luồng minh họa các bước để xác thực token JWT và lấy thông tin người dùng.

- **Bắt đầu:** Quy trình bắt đầu khi nhận yêu cầu lấy thông tin người dùng với `token`.
- **Kiểm tra token:** Kiểm tra xem `token` có `null` không.
 - Nếu `null`, ném ngoại lệ "Invalid token" và kết thúc.
 - Nếu **không null**, tiếp tục.
- **Phân tích token:** Sử dụng `Jwts.parserBuilder()` để phân tích token và lấy `Claims`.
 - Nếu **thành công**, trích xuất tên người dùng (`username`) từ `claims.getSubject()`.
 - Nếu `SignatureException`, ném ngoại lệ "Invalid JWT signature".
 - Nếu `ExpiredJwtException`, ném ngoại lệ "Token expired".
 - Nếu `MalformedJwtException`, ném ngoại lệ "Malformed JWT".
 - Nếu **ngoại lệ khác**, ném ngoại lệ "Failed to validate token".
- **Tìm người dùng:** Gọi `userRepository.findByUsername(username)` để tìm `User`.
 - Nếu **tìm thấy**, ánh xạ `User` thành `UserDTO`.
 - Nếu **không tìm thấy**, ném ngoại lệ "User not found with email: `username`".
- **Trả về UserDTO:** Trả về `UserDTO` chứa thông tin người dùng.
- **Kết thúc:** Kết thúc quy trình.

Quy trình này đảm bảo rằng chỉ token hợp lệ mới được sử dụng để lấy thông tin người dùng, đồng thời xử lý các trường hợp lỗi như token hết hạn hoặc không hợp lệ.



Hình 3.19: Biểu đồ luồng cho quy trình tạo cuộc hội thoại.

3.3 Tạo cuộc hội thoại (Create Conversation)

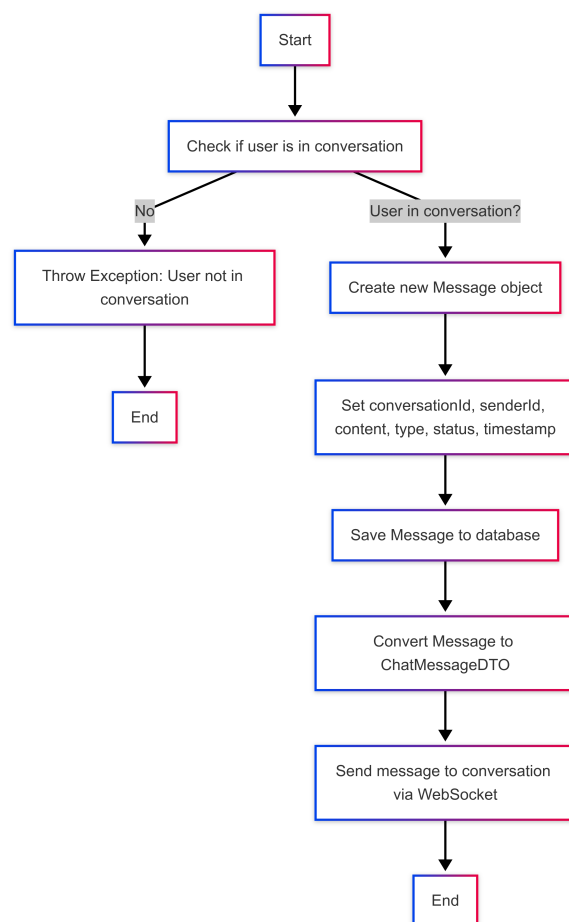
Giải thích quy trình: Quy trình này được thực hiện trong `ChatService.java` (phương thức `createConversation`). Biểu đồ luồng minh họa các bước để tạo một cuộc hội thoại mới và thêm người tham gia.

- **Bắt đầu:** Quy trình bắt đầu khi nhận yêu cầu tạo cuộc hội thoại với `name`, `isGroupChat`, và `participantIds`.
- **Tạo đối tượng cuộc hội thoại:** Tạo một đối tượng `Conversation` mới.
- **Thiết lập thông tin:** Gán `name`, `groupChat`, và `createdAt` (thời gian hiện tại).
- **Lưu cuộc hội thoại:** Gọi `conversationRepository.save` để lưu `Conversation`.
- **Khởi tạo danh sách người tham gia:** Tạo danh sách rỗng `participants`.
- **Vòng lặp cho mỗi `userId`:** Duyệt qua danh sách `participantIds`.
 - **Lấy thông tin người dùng:** Gọi `userClient.getUserById(userId)`.
 - **Tạo người tham gia:** Tạo một `ConversationParticipant` mới.
 - **Thiết lập thông tin:** Gán `conversation`, `userId`, và `joinedAt` (thời gian hiện tại).

- **Thêm vào danh sách:** Thêm `ConversationParticipant` vào `participants`.
- **Lưu tất cả người tham gia:** Gọi `participantRepository.saveAll` để lưu danh sách `participants`.
- **Lấy lại cuộc hội thoại:** Gọi `conversationRepository.findById` để lấy `Conversation` đã lưu.
- **Kết thúc:** Trả về `Conversation` và kết thúc.

Quy trình này đảm bảo rằng cuộc hội thoại được tạo và lưu trữ với đầy đủ thông tin người tham gia.

3.4 Gửi tin nhắn (Send Message)



Hình 3.20: Biểu đồ luồng cho quy trình gửi tin nhắn.

Giải thích quy trình: Quy trình này được thực hiện trong `ChatService.java` (phương thức `sendMessage`). Biểu đồ luồng minh họa các bước để gửi tin nhắn trong một cuộc hội thoại qua WebSocket.

- **Bắt đầu:** Quy trình bắt đầu khi nhận yêu cầu gửi tin nhắn với `conversationId`, `senderId`, `content`, `messageType`, và `mediaUrl`.
- **Kiểm tra người dùng:** Gọi `isUserInConversation(senderId, conversationId)` để kiểm tra xem người gửi có trong cuộc hội thoại không.

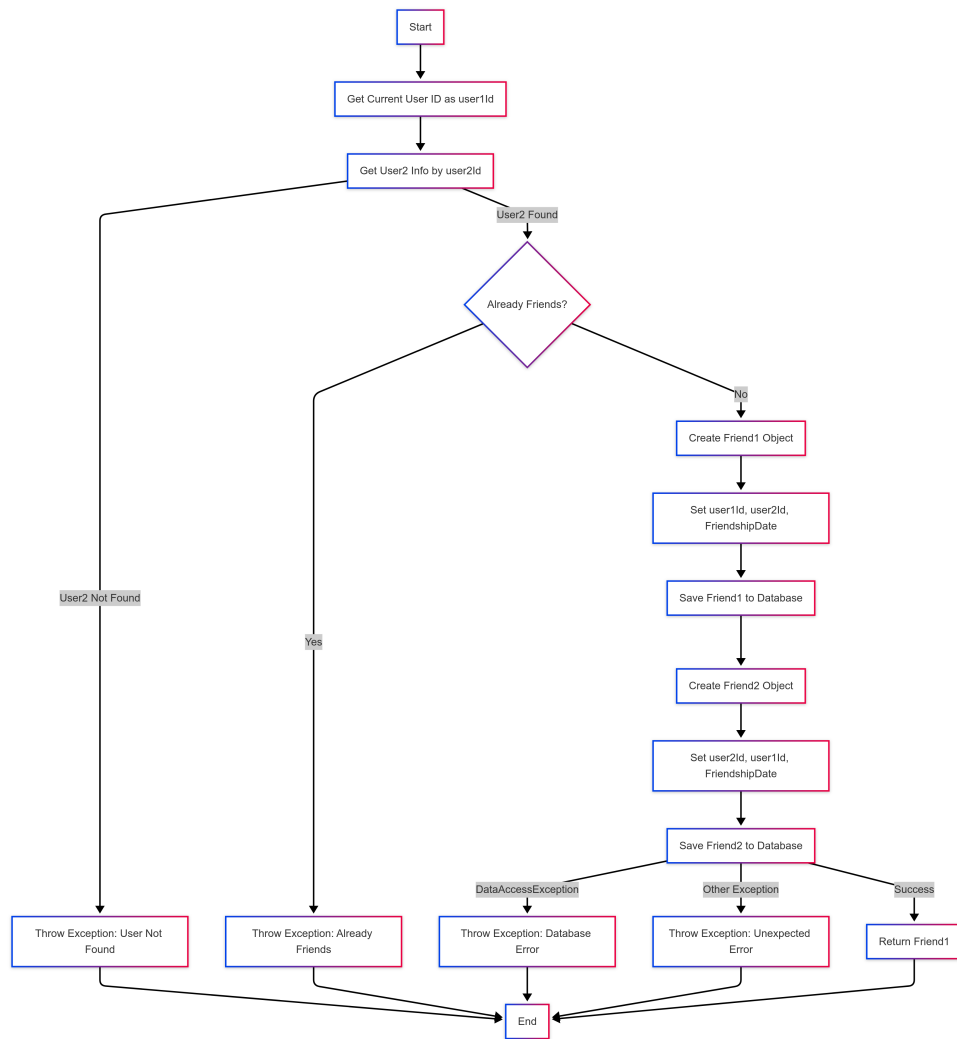
- Nếu **không**, ném ngoại lệ "User is not a participant in this conversation" và kết thúc.
- Nếu **có**, tiếp tục.
- **Tạo đối tượng tin nhắn:** Tạo một đối tượng Message mới.
- **Thiết lập thông tin:** Gán conversationId, senderId, content, type (mặc định là TEXT), status (mặc định là SENT), và timestamp.
- **Lưu tin nhắn:** Gọi messageRepository.save để lưu Message.
- **Chuyển đổi thành DTO:** Chuyển Message thành ChatMessageDTO.
- **Gửi qua WebSocket:** Gọi sendToConversation để phát tin nhắn đến tất cả người tham gia.
- **Kết thúc:** Trả về ChatMessageDTO và kết thúc.

Quy trình này đảm bảo rằng tin nhắn chỉ được gửi khi người gửi thuộc cuộc hội thoại, đồng thời phát tin nhắn qua WebSocket để thông báo thời gian thực.

3.5 Thêm bạn bè (Add Friend)

Giải thích quy trình: Quy trình này được thực hiện trong FriendService.java (phương thức addFriend). Biểu đồ luồng minh họa các bước để thêm mối quan hệ bạn bè giữa hai người dùng.

- **Bắt đầu:** Quy trình bắt đầu khi nhận yêu cầu thêm bạn bè với user2Id.
- **Lấy ID người dùng hiện tại:** Gọi getCurrentUserId() để lấy user1Id.
- **Lấy thông tin người dùng thứ hai:** Gọi userClient.getUserById(user2Id).
 - Nếu **không tìm thấy**, ném ngoại lệ "User with ID user2Id does not exist" và kết thúc.
 - Nếu **tìm thấy**, tiếp tục.
- **Kiểm tra đã là bạn bè chưa:** Gọi friendRepository.existsByUser1IdAndUser2Id để kiểm tra.
 - Nếu **đã là bạn bè**, ném ngoại lệ "You are already friends with user user2Id" và kết thúc.
 - Nếu **chưa là bạn bè**, tiếp tục.
- **Tạo mối quan hệ Friend1:** Tạo một Friend (friend1) từ user1Id đến user2Id.
- **Thiết lập thông tin Friend1:** Gán user1Id, user2Id, và friendshipDate.
- **Lưu Friend1:** Gọi friendRepository.save để lưu friend1.
- **Tạo mối quan hệ Friend2:** Tạo một Friend (friend2) từ user2Id đến user1Id.
- **Thiết lập thông tin Friend2:** Gán user1Id, user2Id, và friendshipDate.



Hình 3.21: Biểu đồ luồng cho quy trình thêm bạn bè.

- **Lưu Friend2:** Gọi `friendRepository.save` để lưu `friend2`.

- Nếu **thành công**, trả về `friend1`.
- Nếu **`DataAccessException`**, ném ngoại lệ "Database error".
- Nếu **ngoại lệ khác**, ném ngoại lệ "Unexpected error".

- **Kết thúc:** Kết thúc quy trình.

Quy trình này đảm bảo rằng mỗi quan hệ bạn bè được lưu trữ hai chiều và chỉ được tạo khi hai người dùng chưa là bạn bè.

Chương 4: Cài đặt và triển khai

I. Yêu cầu hệ thống

Để vận hành ứng dụng Facebook Clone trên máy cá nhân hoặc triển khai lên môi trường máy chủ, hệ thống cần đáp ứng các yêu cầu phần cứng và phần mềm sau:

1. Yêu cầu phần cứng

- **RAM:** Tối thiểu 4GB; khuyến nghị 8GB để đảm bảo hiệu suất khi chạy Docker và các dịch vụ liên quan.
- **CPU:** Tối thiểu 2GHz; khuyến nghị từ 3GHz trở lên để xử lý nhanh các tác vụ back-end và front-end.
- **Dung lượng ổ cứng:** Tối thiểu 10GB để lưu trữ mã nguồn, file âm nhạc và video, và các container Docker.
- **Kết nối mạng:** Cần có kết nối Internet ổn định để tải các dependency và truy cập Cloudinary (lưu trữ file âm nhạc và video).

2. Yêu cầu phần mềm

- **Hệ điều hành:** Windows 10/11, Ubuntu 20.04+, macOS 12+ (hướng dẫn cài đặt sẽ tập trung vào Windows).
- **Phần mềm cần cài:**
 - **Docker Desktop:** Dùng để chạy ứng dụng trong container (phiên bản mới nhất hỗ trợ Windows).
 - **Trình duyệt web:** Chrome (v90+), Firefox (v85+), Edge (v90+) hỗ trợ truy cập giao diện Front-end.

II. Các bước cài đặt

Dưới đây là các bước cài đặt ứng dụng Facebook Clone trên máy cá nhân sử dụng Windows và Docker:

1. Cài đặt Docker Desktop

Truy cập trang chính thức của Docker tại <https://www.docker.com/products/docker-desktop/>. Tải và cài đặt Docker Desktop cho Windows (yêu cầu Windows 10/11 Pro hoặc Enterprise, hoặc bật WSL 2 nếu dùng Windows Home).

Sau khi cài đặt, khởi động Docker Desktop và đảm bảo nó đang chạy (biểu tượng Docker xuất hiện trên thanh taskbar).

2. Clone mã nguồn từ repository

Mở Command Prompt hoặc PowerShell.

Clone repository của dự án bằng lệnh:

```
git clone https://github.com/trucpham04/XDPMTMHPL.git
cd XDPMTMHPL
```

3. Chạy ứng dụng bằng Docker

Mở Command Prompt hoặc PowerShell trong thư mục dự án.

Chạy lệnh sau để build và khởi động ứng dụng:

```
docker-compose up --build
```

Lệnh này sẽ:

- Build các container cho Front-end và Back-end.
- Khởi chạy ứng dụng với cấu hình trong `docker-compose.yml`.

Sau khi chạy thành công có thể truy cập tại: <http://localhost:5173>

Chương 5: Đóng góp của thành viên

I. Phân công nhiệm vụ

Thành viên	Nhiệm vụ chính
Mao Thị Minh Hằng	Post Service (frontend và backend)
Nguyễn Thị Thanh Hằng	Friend Service (frontend và backend)
Trần Bảo Hân	User Service (frontend và backend)
Nguyễn Thùy Trang	Search Service (frontend và backend), hỗ trợ chỉnh sửa frontend, viết báo cáo
Phạm Duy Trực	Message Service (frontend và backend), trang quản trị (admin)

Bảng 5.1: Phân công nhiệm vụ của các thành viên

II. Đóng góp chi tiết của từng thành viên

1. Mao Thị Minh Hằng

Chịu trách nhiệm chính trong việc phát triển Post Service, bao gồm cả frontend và backend. Ở phía backend, Minh Hằng thiết kế các API cho việc tạo, chỉnh sửa, xóa bài viết,... Ở phía frontend, Minh Hằng xây dựng giao diện đăng bài, hiển thị dòng thời gian (timeline), tích hợp các chức năng tương tác như like, comment. Bên cạnh đó, Minh Hằng cũng phối hợp trong việc kiểm thử tích hợp giữa Post Service với các service khác như User Service.

2. Nguyễn Thị Thanh Hằng

Phụ trách phát triển Friend Service, gồm cả frontend và backend. Ở backend, Thanh Hằng xây dựng hệ thống kết bạn, gửi và xử lý yêu cầu kết bạn, danh sách bạn bè. Ở frontend, Thanh Hằng phát triển giao diện danh sách bạn bè, trang yêu cầu kết bạn và xử lý các tương tác giữa người dùng.

3. Trần Bảo Hân

Đảm nhiệm phát triển toàn bộ User Service, bao gồm các chức năng đăng ký, đăng nhập, cập nhật thông tin cá nhân, xác thực người dùng và phân quyền. Backend được thiết kế theo hướng bảo mật cao, sử dụng JWT cho authentication. Frontend được xây dựng với các biểu mẫu tương tác và thân thiện với người dùng.

4. Nguyễn Thùy Trang

Phụ trách xây dựng Search Service, gồm cả frontend và backend. Backend xử lý tìm kiếm người dùng, bài viết, lịch sử tìm kiếm. Bên cạnh đó, Thùy Trang còn chịu trách nhiệm chính trong việc chỉnh sửa và hoàn thiện tài liệu báo cáo.

5. Phạm Duy Trức

Đảm nhận phát triển Message Service và Admin. Với Message Service, Trức xây dựng hệ thống nhắn tin thời gian thực sử dụng WebSocket, bao gồm gửi, nhận, lưu trữ và hiển thị tin nhắn. Ở phần admin, Trức phát triển giao diện quản trị để giám sát hệ thống, quản lý người dùng và bài viết.

Chương 6: Kết luận

I. Tóm tắt dự án

Dự án Facebook Clone được xây dựng nhằm mô phỏng các chức năng cơ bản của một mạng xã hội hiện đại, đồng thời ứng dụng các công nghệ tiên tiến như kiến trúc microservice, Spring Boot, React, Docker, RabbitMQ và WebSocket.

Hệ thống cho phép người dùng:

- Đăng ký, đăng nhập, chỉnh sửa thông tin cá nhân.
- Kết bạn, gửi và chấp nhận lời mời kết bạn.
- Đăng bài viết, bình luận, thích bài viết.
- Tìm kiếm người dùng, lưu và hiển thị lịch sử tìm kiếm.
- Nhắn tin với bạn bè thông qua chức năng chat realtime sử dụng WebSocket.

Hệ thống được xây dựng theo hướng microservice, giúp dễ mở rộng, bảo trì và triển khai. RabbitMQ được sử dụng để xử lý giao tiếp bất đồng bộ giữa các service, trong khi WebSocket đảm nhiệm truyền tải dữ liệu realtime trong chức năng chat. Giao diện người dùng hiện đại được phát triển bằng React kết hợp Redux và các thư viện hỗ trợ.

II. Hạn chế

Mặc dù dự án đã đáp ứng được nhiều chức năng cơ bản của một mạng xã hội, tuy nhiên vẫn tồn tại một số hạn chế:

- Hệ thống chưa hỗ trợ tính năng gọi video, livestream hoặc thông báo đẩy (push notification).
- Chưa hỗ trợ gửi file đính kèm (ảnh, video) trong phần chat.
- Giao diện chưa được tối ưu hoàn toàn cho thiết bị di động.
- Hệ thống chưa tích hợp các biện pháp bảo mật nâng cao như bảo vệ chống XSS, CSRF, hay giới hạn số lần đăng nhập sai.
- Thiếu tính năng xác minh tài khoản qua email hoặc OTP.

III. Cải tiến trong tương lai

Để nâng cao chất lượng và trải nghiệm người dùng, nhóm đề xuất một số hướng cải tiến trong tương lai như sau:

- Tích hợp hệ thống thông báo realtime và thông báo đẩy (push notification).
- Thêm tính năng gửi ảnh, video trong tin nhắn.
- Tích hợp AI để gợi ý bạn bè, lọc nội dung không phù hợp, phân tích hành vi người dùng.
- Tối ưu bảo mật hệ thống: mã hóa dữ liệu nhạy cảm, xác thực đa yếu tố (MFA), và kiểm tra lỗ hổng bảo mật.
- Phát triển phiên bản mobile app bằng React Native hoặc Flutter.